

Tập luận văn đội tuyển quốc gia Trung Quốc năm 2015

Huấn luyện viên: Yu Linyun

China Computer Federation, 2015

Nguồn gốc: Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2015

IOI China candidate team papers / National training team 2015 collection.pdf

Tóm tắt

PDF nguồn là tập hợp luận văn đội tuyển quốc gia Trung Quốc năm 2015. Tập được tạo bằng pdfTeX nhưng lớp văn bản không ánh xạ đúng về Unicode: ngay cả mục lục khi trích xuất bằng pdftotext cũng trở thành mojibake. Bản dịch này chuyển ngữ phần đầu sách, mục lục đọc được từ các trang đã render, và bài đầu tiên của tập sách, được OCR từ ảnh trang.

1 Thông tin đầu sách

Tập sách có tiêu đề *Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2015*. Huấn luyện viên ghi trên bìa là Yu Linyun. Đơn vị xuất bản là China Computer Federation.

2 Mục lục đã dịch

Trang	Bài viết	Tác giả / trường
1	Mở rộng suffix automaton trên trie	Liu Yanyi, Trường Yali, Trường Sa, Hồ Nam
17	Bàn về ứng dụng tư tưởng heuristic trong thi tin học	Ren Zhizhou, Trường Trung học số 1 Thiệu Hưng
37	Bàn về một số phương pháp khớp chuỗi	Wang Shenghao, Trường Trung học số 1 Thiệu Hưng
67	Suffix automaton và ứng dụng	Zhang Tianyang, Trường Trung học số 1 Trường Sa, Hồ Nam
81	Phép toán trên hàm sinh và bài toán đếm tổ hợp	Jin Ce, Trường Học Quân Hàng Châu, Chiết Giang
103	Báo cáo ra đề <i>yc's bonus</i>	Liu Jiancheng, Trường Yali, Trường Sa
113	Bàn về ứng dụng chia khối trong một lớp bài toán trực tuyến	Zou Daoyao, Trường Trung học Trần Hải, Ninh Ba
129	Thuật toán liên quan tới cactus và ứng dụng	Wang Yisong, Trường Học Quân Hàng Châu, Chiết Giang
149	Bàn về thuật toán matching trên đồ thị và ứng dụng	Chen Haobo, Trường Trường Quận, Trường Sa, Hồ Nam
175	Bàn về các bài toán vật lý trong thi tin học	Chen Siyu, Trường Trung học số 2 Hàng Châu
199	Báo cáo ra đề <i>Đề bài bị thất lạc</i>	Yu Jiping, Trường Trung học số 24 Đại Liên
205	Một số kỹ thuật tối ưu DP	Zhang Hengjie, Trường Trung học số 1 Thiệu Hưng
217	Báo cáo ra đề <i>Product</i>	Du Yuhao, Trường Trung học Trần Hải, Ninh Ba
231	Bước đầu tìm hiểu một lớp bài toán lấy mã nguồn làm đầu vào	Lu Xiaochen, Học viện Giao Xuyên Trần Hải, Ninh Ba
271	Tính chất, ứng dụng và thuật toán nhanh cho lũy thừa tập hợp	Lu Kaifeng, Trường Trung học số 2 Vũ Hán, Hồ Bắc

3 Mở rộng suffix automaton trên trie

Liu Yanyi

Trường Yali, Trường Sa, Hồ Nam

Tóm tắt

Bài viết phân tích chi tiết một automaton hữu hạn xác định có thể nhận ra mọi hậu tố của một trie. Tác giả đưa ra và chứng minh các kết luận rằng số trạng thái là tuyến tính, tổng số hàm chuyển không vượt quá số nút của trie nhân với kích thước bảng chữ cái, đồng thời trình bày một thuật toán xây dựng khả thi, có độ phức tạp tuyến tính khi được phép xử lý offline. Bài viết cũng phân tích độ phức tạp của thuật toán xây dựng online, giới thiệu một số ứng dụng liên quan của suffix automaton, với hy vọng gợi mở thêm các hướng sử dụng cấu trúc này.

3.1 Lời nói đầu

Trong thi thuật toán, xử lý xâu là một mảng rất quan trọng. Nhìn chung có hai loại bài toán lớn: bài toán có mẫu cố định và bài toán có văn bản chính cố định. Khi mẫu cố định, có thể có nhiều văn bản chính khác nhau. Nếu chỉ có một mẫu, thuật toán KMP nổi tiếng [2, 3] xử lý được lớp bài toán này; cũng có thể dùng hashing xâu [13] để đếm số lần xuất hiện của mẫu. Nếu có nhiều mẫu, thuật toán AC automaton quen thuộc [1] đảm nhiệm tốt lớp bài toán đó. Nếu văn bản chính cố định, ta thường xây suffix array [11], suffix tree [4, 5] hoặc suffix automaton [7] cho văn bản chính. Lớp bài toán này đã được nghiên cứu đầy đủ trong cộng đồng thi thuật toán. Chẳng hạn, Chen Lijie đã nêu nhiều ứng dụng của suffix automaton trên một xâu trong bài giảng giao lưu trại đông [8], còn Luo Suijian giải thích chi tiết việc dùng suffix array trong thi tin học trong luận văn đội tuyển quốc gia của mình [12]. Khi đề bài cho phép xử lý offline, hai loại bài toán trên thường có thể chuyển đổi cho nhau, thể hiện tính linh hoạt của các thuật toán xử lý xâu.

Trong kỳ chọn đội tuyển tỉnh Chiết Giang năm 2015 xuất hiện bài toán sau: cho một cây có $n \leq 10^5$ nút, số lá không quá 20, mỗi cạnh mang một ký tự. Đường đi giữa hai nút trên cây tạo thành một xâu; hỏi có tất cả bao nhiêu xâu con khác nhau.¹ Bài này rõ ràng thuộc loại văn bản chính cố định, nhưng văn bản chính không còn là một xâu mà là một cây. Nếu trực tiếp liệt kê mọi cặp đỉnh rồi dùng trie [1, 13] để kiểm tra hai xâu có bằng nhau hay không, độ phức tạp là $O(n^3)$. Ngay cả dùng hashing xâu [13], thời gian vẫn là $O(n^2)$, không thể qua được dữ liệu.

Ta chuyển hóa bài toán như sau. Vì chỉ có 20 lá, với mỗi lá làm gốc, dựng một trie riêng rồi hợp nhất tất cả các trie này thành một trie mới. Khi đó mọi xâu xuất hiện trên cây ban đầu chắc chắn xuất hiện trong trie mới; văn bản chính đã trở thành một trie.

Để giải quyết vấn đề trên, bài viết phân tích automaton hữu hạn xác định nhận ra toàn bộ hậu tố của một trie và đề xuất một cách xây dựng. Nội dung chính gồm năm phần:

1. đưa một số khái niệm trên xâu sang trie, đồng thời giới thiệu suffix automaton của trie;
2. phân tích suffix link và xây dựng cấu trúc cây parent;
3. chứng minh số trạng thái là tuyến tính và nêu chặn trên cho tổng số hàm chuyển trong automaton;
4. mô tả thuật toán xây suffix automaton cho trie; khi được phép offline, độ phức tạp của nó là tuyến tính theo kích thước automaton;
5. trình bày một số ứng dụng của suffix automaton trên trie, qua đó làm rõ hơn cách nhìn về suffix automaton.

3.2 Suffix automaton của trie

Trong bài viết này, mọi ký tự của xâu và mọi ký tự trên cạnh của trie đều thuộc một bảng chữ cái hữu hạn A . Tập mọi xâu sinh bởi bảng chữ cái này là A^* ; xâu rỗng được ký hiệu là ε . Tập không chứa xâu rỗng là $A^+ = A^* \setminus \{\varepsilon\}$. Dùng $|s|$ để chỉ độ dài xâu s , s_i là ký tự thứ i , và $s_{i,j}$ là xâu gồm các ký tự từ vị trí i đến vị trí j của s . Ký hiệu $a \cdot b$ là xâu thu được khi nối a với b ; dấu nhân có thể được lược bỏ.

Tập mọi hậu tố của một xâu s là

$$S(s) = \{s_{i,|s|} \mid 1 \leq i \leq |s|\}.$$

Với một tập xâu S , $Minl(S)$ và $Maxl(S)$ lần lượt là độ dài của xâu ngắn nhất và dài nhất trong S .

¹ZJOI 2015 Day 1, *Vùng đất huyền tưởng được chut thần ưu ái*, tác giả Chen Lijie.

Dùng T để chỉ một trie, và dùng $T_x, T_y, T_z, \dots$ để chỉ các nút của T . Xây nhận được bằng cách nối theo thứ tự các ký tự trên đường đi từ nút T_x đến một nút T_y trong cây con của nó được ký hiệu là $T_{x,y}$. Nếu không nói rõ, khi dùng $T_{x,y}$, ta luôn ngầm hiểu T_y nằm trong cây con của T_x .

Một tiền tố của T là

$$P_{T_x} = T_{\text{root},x} \quad (\text{root là gốc của } T, T_x \in T).$$

Tập mọi xâu con của T là

$$F(T) = \{T_{x,y} \mid T_x, T_y \in T, T_y \text{ nằm trong cây con của } T_x\}.$$

Tương tự, tập mọi hậu tố của T là

$$S(T) = \{T_{x,y} \mid T_x, T_y \in T, T_y \text{ là nút lá}\}.$$

Xây suffix automaton cho trie nghĩa là xây một automaton hữu hạn có thể nhận ra mọi xâu trong $S(T)$, đồng thời cố gắng giảm số trạng thái và số chuyển xuống càng nhỏ càng tốt. Tương tự suffix automaton trên một xâu, ta định nghĩa vị trí xuất hiện của một xâu trên trie:

$$\text{Right}(s) = \{T_y \mid T_y \in T, \exists T_x \in T, T_y \text{ nằm trong cây con của } T_x, T_{x,y} = s\}.$$

Định nghĩa quan hệ tương đương R_T ; khi không gây nhầm lẫn viết là R :

$$aRb \iff \text{Right}(a) = \text{Right}(b) \quad (a, b \in A^*).$$

Gọi $R(s)$ là tập mọi xâu tương đương với s , tức một lớp tương đương. Trong automaton, mỗi trạng thái tương ứng với một lớp tương đương.

Gọi suffix automaton xây từ T là $\Phi(T)$, trạng thái đầu là $\Phi(T)_\varepsilon$. Ký hiệu $\Phi(T)_s$, với $s \in F(T)$, là trạng thái sau khi đọc s . Hàm chuyển được định nghĩa bởi chuyển nhân a từ $\Phi(T)_s$ tới $\Phi(T)_{sa}$ khi $a \in A$ và $sa \in F(T)$. Khi nối hai xâu s_1, s_2 mà $s_1, s_2, s_1s_2 \in F(T)$, ta cũng có $\Phi(T)_{s_1}$ đọc s_2 tới $\Phi(T)_{s_1s_2}$. Theo định nghĩa,

$$\Phi(T)_a = \Phi(T)_b \iff aRb \quad (a, b \in F(T)).$$

Nếu a là một xâu tương ứng với trạng thái $\Phi(T)_b$, thì tập xâu tương ứng với trạng thái $\Phi(T)_b$ chính là $R(b)$.

Hình a. Trong trie minh họa của bản gốc, b tương đương với ab ; các nút màu đỏ là tập Right của chúng.

Để phân tích số trạng thái của toàn bộ cấu trúc, tức số lớp tương đương R trong $F(T)$, ta cần xét một số tính chất của các trạng thái suffix automaton. Mục tiêu là làm số trạng thái nhỏ nhất có thể.

Bổ đề 2.1. Với $x, y, z, u \in F(T)$, ta có:

- (i) $xRy \Rightarrow (x \in S(y) \text{ hoặc } y \in S(x))$,
- (ii) $x \in S(z) \Rightarrow \text{Right}(z) \subseteq \text{Right}(x)$,
- (iii) $(x \in S(z), z \in S(u), xRu) \Rightarrow zRuRx$.

Chứng minh. (i) Không mất tính tổng quát, giả sử $|x| \leq |y|$ và $Right(x) = Right(y)$. Lấy tùy ý $T_v \in Right(x)$. Khi đó $x \in S(P_{T_v})$ và $y \in S(P_{T_v})$, nên $x \in S(y)$.

(ii) Với mọi $T_v \in Right(z)$, ta có $z \in S(P_{T_v})$. Vì $x \in S(z)$, suy ra $x \in S(P_{T_v})$, do đó $T_v \in Right(x)$. Vậy $Right(z) \subseteq Right(x)$.

(iii) Từ (ii), $z \in S(u)$ cho $Right(u) \subseteq Right(z)$, và $x \in S(z)$ cho $Right(z) \subseteq Right(x)$. Lại có xRu , nên $Right(z) = Right(u) = Right(x)$, tức $zRuRx$.

Mệnh đề (i) cũng cho thấy trong cùng một lớp tương đương R , không thể có hai xâu khác nhau nhưng cùng độ dài.

Hệ quả 2.2. Với một lớp tương đương $R(a)$, gọi x là xâu dài nhất trong $R(a)$, và y là xâu ngắn nhất trong $R(a)$. Khi đó

$$R(a) = \{z \mid z \in S(x) \text{ và } |y| \leq |z| \leq |x|\}.$$

Điều này suy ra trực tiếp từ Bổ đề 2.1.

3.3 Suffix link

Trong thuật toán xây suffix automaton cho một xâu, suffix link giữ vai trò then chốt [7, 6], và toàn bộ thuật toán rất giống KMP [3]. Khi xây suffix automaton cho trie, ta cũng giữ suffix link và so sánh quá trình xây dựng với thuật toán xây automaton tối thiểu xác định nhận ra tập $A^*\{P_{T_x}\}$, tức cách xây AC automaton quen thuộc [1]. Tương tự hàm fail trong AC automaton, ta định nghĩa suffix link của suffix automaton trên trie như sau.

Suffix link là một hàm $s_T : A^* \setminus \{\varepsilon\} \rightarrow A^*$, khi không nhầm lẫn viết là s :

$$s_T(x) = \{z \mid z \in S(x), |z| = \text{Minl}(R_T(x)) - 1\}.$$

Nói cách khác, đó là hậu tố dài nhất y của x sao cho xR_Ty không đúng.

Bổ đề 3.1. Với $x, y \in F(T)$, ta có:

- (i) $xRy \Rightarrow s(x) = s(y)$,
- (ii) $Right(x) \subset Right(s(x))$,
- (iii) $\text{Minl}(R(x)) - 1 = |s(x)| = \text{Maxl}(R(s(x)))$.

Chứng minh. (i) Không mất tính tổng quát, giả sử $|x| \leq |y|$. Theo định nghĩa, $|s(x)| = \text{Minl}(R(x)) - 1$, $|s(y)| = \text{Minl}(R(y)) - 1$, mà $R(x) = R(y)$, nên $|s(x)| = |s(y)| < |x| \leq |y|$. Theo Bổ đề 2.1(i), $x \in S(y)$. Lại có $s(x) \in S(x)$ và $s(y) \in S(y)$, do đó $s(x) = s(y)$.

(ii) Theo Bổ đề 2.1(ii), $Right(x) \subseteq Right(s(x))$. Vì $xRs(x)$ là sai, hai tập này không bằng nhau, nên bao hàm là nghiêm ngặt.

(iii) Vì $s(x) \in R(s(x))$, ta có $|s(x)| \leq \text{Maxl}(R(s(x)))$. Giả sử tồn tại xâu y tương đương với $s(x)$ và dài nhất trong lớp đó. Khi ấy $Right(y) = Right(s(x))$, đồng thời $s(x) \in S(y)$. Dùng phản chứng: nếu $|y| > |s(x)|$, lấy $T_v \in Right(x)$. Từ (ii), $T_v \in Right(s(x))$, suy ra $T_v \in Right(y)$, nên $y \in S(P_{T_v})$ và $x \in S(P_{T_v})$. Nếu $|y| > |x|$ thì $x \in S(y)$, theo Bổ đề 2.1(ii) sẽ có $Right(s(x)) = Right(y) \subseteq Right(x)$, mâu thuẫn với (ii). Nếu $|y| \leq |x|$, vì xRy không đúng nên $s(x) = y$, mâu thuẫn với $|y| > |s(x)|$. Vậy mệnh đề được chứng minh.

Từ (i), mọi xâu trong cùng một lớp tương đương $R(a)$ có suffix link trở tới cùng một xâu. Vì vậy ta có thể gán suffix link cho từng trạng thái của suffix automaton. Cụ thể, suffix link của $R(a)$ trở tới $R(s(a))$.

Theo (ii) hoặc (iii), các suffix link này tạo thành một cây; ta gọi nó là **cây parent**. Định nghĩa $Fa(\Phi(T)_s)$ là suffix link của trạng thái $\Phi(T)_s$, tức

$$Fa(\Phi(T)_a) = \Phi(T)_b \iff s(a)Rb.$$

Bổ đề 3.2. Với $x \in F(T)$, lấy tùy ý $T_v \in \text{Right}(x)$. Nếu $|P_{T_v}| > \text{Maxl}(R(x))$ và đặt $a = P_{T_v}$, thì

$$s(a_{|a|-\text{Maxl}(R(x)), |a|})Rx.$$

Chứng minh. Đặt $b = a_{|a|-\text{Maxl}(R(x)), |a|}$. Từ giả thiết có $x \in S(b)$. Theo Hệ quả 2.2, vì $|b| > \text{Maxl}(R(x))$, nên bRx không đúng, còn hậu tố độ dài $\text{Maxl}(R(x))$ của b lại tương đương với x . Vì vậy b không tương đương với hậu tố đó; theo định nghĩa suffix link, $s(b)$ chính là hậu tố này, và $s(b)Rx$.

Hình b. Cây trong bản gốc là cây parent của suffix automaton được xây từ trie ở Hình a.

3.4 Số trạng thái tuyến tính

Với suffix automaton xây cho một xâu x , kích thước automaton là $O(|x|)$ [7, 6]. Ta cũng mong suffix automaton xây cho trie có quy mô trạng thái tuyến tính. Sau khi định nghĩa suffix link, cần phân tích kỹ hơn $\Phi(T)$.

Bổ đề 4.1. Với $x, y \in F(T)$, ta có:

- (i) $\forall y, \neg(xRs(y)) \Rightarrow |\text{Right}(x)| = 1$,
- (ii) $|\text{Right}(x)| = 1 \Rightarrow P_{T_v}Rx \quad (\text{Right}(x) = \{T_v\})$,
- (iii) $\forall T_v \in \text{Right}(x), \neg(xRP_{T_v})$
 $\Rightarrow \exists u, v, s(u)Rs(v)Rx$ và $\neg(uRv)$.

Chứng minh. (i) Chứng minh bằng phản chứng. Giả sử $|\text{Right}(x)| > 1$. Lấy $T_{v_1}, T_{v_2} \in \text{Right}(x)$ và đặt $a = P_{T_{v_1}}, b = P_{T_{v_2}}$. Khi đó $|a|, |b| \geq \text{Maxl}(R(x))$; nếu một trong hai độ dài nhỏ hơn $\text{Maxl}(R(x))$, xâu trong $R(x)$ có độ dài $\text{Maxl}(R(x))$ không thể xuất hiện ở nút tương ứng, mâu thuẫn. Nếu $a \notin R(x)$, theo Bổ đề 3.2 suy ra suffix link của một hậu tố thích hợp của a trở vào lớp của x . Nếu $a \in R(x)$, vì $T_{v_1} \neq T_{v_2}$, ta có $a \neq b$, và $|b| \geq \text{Maxl}(R(x))$, nên một lập luận tương tự cũng cho một xâu có suffix link trở vào $R(x)$. Mệnh đề được chứng minh.

(ii) Biết $x \in S(P_{T_v})$, do đó $|\text{Right}(P_{T_v})| \leq |\text{Right}(x)|$. Vì $|\text{Right}(x)| = 1$, suy ra $\text{Right}(P_{T_v}) = \{T_v\} = \text{Right}(x)$, tức $P_{T_v}Rx$.

(iii) Từ phản đảo của (i), nếu $|\text{Right}(x)| \neq 1$ thì $|\text{Right}(x)| \geq 2$. Theo giả thiết, với mọi $T_v \in \text{Right}(x)$ ta có $|P_{T_v}| > \text{Maxl}(R(x))$. Tồn tại T_{v_1}, T_{v_2} sao cho hậu tố chung dài nhất của $P_{T_{v_1}}$ và $P_{T_{v_2}}$ có độ dài đúng bằng $\text{Maxl}(R(x))$. Nếu không, mọi cặp đều có hậu tố chung dài hơn $\text{Maxl}(R(x))$; theo Bổ đề 2.1(iii), hậu tố chung dài hơn đó cũng tương đương với x , mâu thuẫn với định nghĩa $\text{Maxl}(R(x))$.

Đặt

$$a = (P_{T_{v_1}})_{|P_{T_{v_1}}|-\text{Maxl}(R(x)), |P_{T_{v_1}}|}, \quad b = (P_{T_{v_2}})_{|P_{T_{v_2}}|-\text{Maxl}(R(x)), |P_{T_{v_2}}|}.$$

Từ cách chọn, $a \neq b$, $a \notin S(b)$, $b \notin S(a)$. Theo Bổ đề 2.1(i), $\neg(aRb)$. Dùng Bổ đề 3.2 ta có $s(a)Rx$ và $s(b)Rx$. Lấy $u = a$, $v = b$ là được.

Định lý 4.2. Suffix automaton xây cho một trie có số trạng thái tuyến tính.

Chứng minh. Xét cây parent tạo bởi suffix link. Từ Bổ đề 4.1(i) và (ii), với mọi lá $\Phi(T)_s$ trên cây parent đều tồn tại một tiền tố P_{T_v} sao cho $P_{T_v}Rs$. Từ Bổ đề 4.1(iii), mọi trạng thái $\Phi(T)_s$ mà với mọi T_v đều không có sRP_{T_v} thì phải có ít nhất hai con trong cây parent. Nói cách khác, các lá và các nút chỉ có một con trên cây parent đều tương ứng với một tiền tố nào đó P_{T_v} ; các nút còn lại có ít nhất hai con.

Số tiền tố chỉ là $|T|$, và trạng thái đạt được khi đọc từng P_{T_v} là duy nhất. Vì vậy tổng số lá và nút một con trên cây parent không vượt quá $|T|$; toàn bộ số nút của cây không vượt quá $2|T|$. Do đó số trạng thái là tuyến tính.

Nếu không chỉ số trạng thái mà tổng số hàm chuyển, tức số cạnh, cũng tuyến tính, thì toàn bộ automaton sẽ tuyến tính. Tuy nhiên, khác với suffix automaton của một xâu, tổng số hàm chuyển của suffix automaton trên trie có thể ở cấp số trạng thái nhân với kích thước bảng chữ cái. Ta chứng minh nó không vượt quá cấp này, và đưa ra một trie đạt cùng cấp độ.

Định lý 4.3. Với một trie, chặn trên của số hàm chuyển trong suffix automaton là

$$O((\text{số nút của trie}) \cdot (\text{kích thước bảng chữ cái})).$$

Chứng minh. Gọi trie là T . Theo Định lý 4.2, $|\Phi(T)| \leq 2|T|$. Với mỗi trạng thái $\Phi(T)_s$, số chuyển nhiều nhất là $|A|$. Vì vậy tổng số hàm chuyển không vượt quá $2|T||A|$.

Ta cũng dựng được một trie khiến số hàm chuyển đạt chặn này theo cấp độ. Chỉ cần xét những ký tự thực sự xuất hiện trên cạnh của T , nên $|A| \leq |T|$. Một phần của T là một dây dài $|T|/2$, mọi cạnh trên dây đều mang cùng ký tự a . Các nút còn lại nối từ nút cuối của dây bằng các ký tự đôi một khác nhau; gọi tập ký tự này là B . Khi đó $a^ib \in S(T)$ với $0 \leq i \leq |T|/2$ và $b \in B$, đồng thời hiển nhiên $\neg(a^iRa^j)$ nếu $i \neq j$. Vì thế với mỗi trạng thái $\Phi(T)_{a^i}$, mọi $b \in B$ đều tạo một hàm chuyển. Tổng số hàm chuyển đạt $|T|/2 \cdot |B|$, tức cùng cấp với $|T|/2 \cdot |A|/2$, trùng cấp với chặn trên trong Định lý 4.3.

Hình c. Ví dụ trie trong bản gốc gồm một dây toàn ký tự a , rồi tỏa ra nhiều cạnh ký tự khác nhau, làm tổng số hàm chuyển đạt chặn trên theo cấp độ.

Dù số chuyển khi xây suffix automaton cho trie không luôn tuyến tính, trong phần lớn đề thi và bài toán thực tế, bảng chữ cái không lớn; thường chỉ là chữ cái Latin thường hoặc chữ số. Khi bảng chữ cái là hằng số, số chuyển vẫn tuyến tính, nên đây vẫn là một phương pháp hiệu quả để xử lý các bài toán xâu liên quan đến trie.

3.5 Thuật toán xây dựng

Trong thuật toán xây suffix automaton cho xâu [7, 8], ta dùng một thuật toán tăng dần online: từ suffix automaton của s , xây suffix automaton của sa , với $a \in A$. Khi xây cho trie, ta cũng xét thuật toán tăng dần online. Từ phân tích số trạng thái và số chuyển ở trên, ta biết cận dưới độ phức tạp của thuật toán xây dựng là $|T||A|$.

Giả sử từ trie T , ta thêm vào nút T_v một cạnh nhãn a đi tới nút con mới T_w , thu được trie mới T_N . Đặt $s = P_{T_v}$, nên tiền tố mới là $sa = P_{T_w}$. Xét những phần của $\Phi(T_N)$ cần sửa trên nền $\Phi(T)$. Trước hết phân tích số trạng thái.

3.5.1 Thay đổi trạng thái

Với mọi xâu đã có thể tiếp tục mở rộng thành hậu tố của trie mới, trạng thái tương ứng trong $\Phi(T_N)$ phải tồn tại. Ngược lại, nếu một xâu không thể mở rộng thành hậu tố mới, trạng thái đó không cần xuất hiện thêm. Nếu một lớp tương đương không đổi sau khi thêm T_w , trạng thái suffix automaton tương ứng cũng không đổi, tức $\Phi(T_N)_x = \Phi(T)_x$.

Trường hợp 1. Nếu T_v đã có một con nối bằng cạnh nhãn a , ta không cần sửa $\Phi(T)$.

Trường hợp 2. Nếu $sa \in F(T)$, nghĩa là trong $\Phi(T)$ đã có chuyển nhãn a từ trạng thái của s , thì lớp tương đương $R_T(sa)$ có thể bị tách làm hai lớp.

Thật vậy, với $x, y \in F(T)$ và $xR_T y$, nếu trong tập $Right_{T_N}$ của x và y đều xuất hiện nút mới T_w , thì khi bỏ T_w ra, hai tập vẫn bằng nhau, nên chắc chắn $xR_{T_N} y$. Vì vậy mọi xâu tương đương với sa trong T_N vốn đều nằm trong cùng lớp $R_T(sa)$. Với $x \in R_T(sa)$, nếu $xR_{T_N} sa$, không có trạng thái mới. Nếu tồn tại $x \in R_T(sa)$ nhưng $\neg(xR_{T_N} sa)$, cần tách $R_T(sa)$ thành hai lớp: một lớp tương đương với sa trong T_N , một lớp không tương đương với nó. Lớp không tương đương có tập $Right$ không chứa T_w , nên tập $Right$ của nó giữ nguyên như trong T .

Xét suffix link của hai trạng thái mới. Gọi $u, v \in F(T)$, trong đó $uR_{T_N} sa$, $\neg(vR_{T_N} sa)$, nhưng $uR_T sa$ và $vR_T sa$. Nếu đặt u vào lớp thứ nhất và v vào lớp thứ hai, vì $sa = P_{T_w}$ và $uR_{T_N} sa$, mọi hậu tố của u nhận thêm cùng vị trí mới nên suffix link của lớp thứ nhất không đổi. Với v , phần mới bị tách ra đúng tại lớp của sa , do đó suffix link của lớp thứ hai trở tới lớp mới chứa sa . Nói ngắn gọn, nếu một trạng thái bị tách, một bản sao giữ suffix link cũ, bản sao còn lại có suffix link trở tới trạng thái của sa .

Hình d. Suffix automaton của trie trong Hình a.

Hình e, f, g. Sau khi thêm một nút vào trie của Hình a, bản gốc minh họa lần lượt trie mới, suffix automaton mới và cây parent mới. Nút và cạnh màu cam biểu thị phần mới trong automaton/cây parent.

Khi đã biết lớp tương đương chứa sa trong T_N , mọi hậu tố của sa trước đó đã được đưa vào các lớp liên quan, và các trạng thái ấy không đổi, nên không cần xét lại.

Trường hợp 3. Nếu $sa \notin F(T)$, ta tạo một lớp mới $R_{T_N}(sa)$. Vì xuất hiện một hậu tố mới, vẫn có khả năng một lớp tương đương cũ bị tách.

Có một kết luận khá rõ ràng: $s_{T_N}(sa)$ bằng hậu tố dài nhất của sa có xuất hiện trong $F(T)$. Chứng minh đơn giản: nếu $x \in S(sa)$ nhưng $x \notin F(T)$, thì x chỉ xuất hiện tại nút mới T_w , và sa cũng chỉ xuất hiện tại T_w ; do đó $Right(sa) = Right(x) = \{T_w\}$, nên $xRsa$. Ngược lại, nếu $x \in S(sa)$ và $x \in F(T)$, thì $|Right(x)| > 1$, nên $\neg(xRsa)$.

Kết luận này giúp quá trình xây dựng thuận tiện hơn. Các hậu tố

$$y \in S(sa), \quad y \notin S(s_{T_N}(sa))$$

tạo thành lớp mới $R_{T_N}(sa)$, và ta tạo một trạng thái mới cho lớp đó. Với những $y \in S(s_{T_N}(sa))$, tức phần hậu tố còn lại, có thể xem $s_{T_N}(sa)$ như sa trong Trường hợp 2. Dù Trường hợp 2 dùng tính chất $sa = P_{T_w}$, bản thân $s_{T_N}(sa)$ đã có tính cực đại cần thiết, nên không gây sai lệch.

Bài toán còn lại là tìm $s_{T_N}(sa)$. Theo kết luận trên, ta cần hậu tố dài nhất của sa đã xuất hiện ít nhất một lần trong $F(T)$. Bắt đầu từ trạng thái $\Phi(T)_s$, đi lên cây parent qua các suffix link cho đến khi gặp trạng thái đầu tiên có chuyển nhãn a . Nếu trạng thái đó đại diện bởi một xâu r , thì ra là hậu tố dài nhất của sa xuất hiện ít nhất hai lần, tức $s_{T_N}(sa) = ra$.

Từ ba trường hợp trên, khi thêm một ký tự vào trie, số trạng thái tăng nhiều nhất hai. Điều này một lần nữa chứng minh số trạng thái tuyến tính, đồng thời mô tả được thay đổi của suffix link. Ta vẫn chưa xét cách sửa hàm chuyển; phần tiếp theo xử lý điểm này.

3.5.2 Thay đổi hàm chuyển

Trong Trường hợp 2, nếu không tạo trạng thái mới, chắc chắn không có thay đổi hàm chuyển. Nếu có, lớp cũ $R_T(sa)$ bị tách thành hai lớp $R_{T_N}(sa)$ và một lớp còn lại. Vì nút mới T_w là lá, nó không làm thay đổi các chuyển vốn có từ các xâu cũ. Các chuyển của hai trạng thái mới có thể sao chép từ trạng thái cũ tương ứng.

Tuy nhiên, vì lớp $R_T(sa)$ cũ bị xóa, có thể tồn tại các trạng thái có chuyển nhãn a trở tới trạng thái cũ đó; những chuyển này phải được sửa. Theo phân tích ở Trường hợp 2, các chuyển dọc theo chuỗi suffix link từ trạng thái của s lên trên, chừng nào chúng còn đi tới lớp cũ, sẽ được đổi để trở tới lớp mới phù hợp. Những chuyển khác giữ nguyên. Cách làm trực tiếp là giữ một bản sao đại diện cho phần không đổi, tạo một trạng thái mới cho phần bị tách, rồi bạo lực sửa các chuyển nhãn a trên chuỗi suffix link cho tới khi gặp trạng thái mà chuyển nhãn a không còn trở tới lớp cũ.

Trong Trường hợp 3, ta tạo trạng thái mới $R_{T_N}(sa)$; phần còn lại giống Trường hợp 2. Vì T_w là lá, trạng thái mới ban đầu không có chuyển ra. Do có trạng thái mới, một số trạng thái trên chuỗi suffix link có thể cần thêm chuyển nhãn a trở tới trạng thái mới của sa . Theo phân tích Trường hợp 3, chỉ cần thêm các chuyển này dọc theo chuỗi từ trạng thái của s đi lên cho tới khi gặp trạng thái đã có chuyển nhãn a thích hợp.

3.5.3 Phân tích độ phức tạp

Tương tự trường hợp một xâu, toàn bộ thuật toán là thuật toán tăng dần online. Với trie T , số lần thực hiện là $|T|$. Mỗi lần chỉ thêm nhiều nhất hai trạng thái, nên phần trạng thái là $O(|T|)$. Suffix link của mỗi trạng thái mới chỉ sửa một lần, phần này cũng là $O(|T|)$.

Xét hàm chuyển. Trong Trường hợp 2 có thể cần sao chép một tập chuyển; mỗi trạng thái có nhiều nhất $|A|$ chuyển, nên phần này là $O(|T| |A|)$. Phần phức tạp hơn là các lần bạo lực sửa chuyển khác. Có thể chứng minh tổng thời gian cho phần này không vượt quá tổng độ sâu các lá trong trie; gọi tổng đó là $G(T)$.

Ý tưởng chứng minh giống phân tích thế năng. Gọi $Dep(y)$, với $y \in F(T)$, là độ sâu của trạng thái $\Phi(T)_y$ trong cây parent. Xét đường từ trạng thái $\Phi(T)_s$ và trạng thái mới tới gốc trên cây parent. Theo Bổ đề 2.1(iii), nếu tồn tại lớp của sa thì cũng tồn tại lớp của s , nên $Dep(sa) \leq Dep(s) + 1$. Khi bạo lực sửa chuyển, ta liên tục đặt $c = s(c)$ và sửa chuyển của c ; mỗi bước như vậy làm $Dep(c)$ giảm 1. Khi dừng, độ sâu của trạng thái liên quan chỉ có thể tăng thêm một hằng số so với $Dep(s)$. Vì mỗi lần sửa đều làm giảm thế năng, tổng số lần sửa là cấp độ độ sâu.

Có thể xem Dep là hàm thế năng của trạng thái hiện tại. Với mỗi lá của T , nếu thực hiện riêng quá trình chèn theo đường từ gốc tới lá đó, mỗi phép chèn làm Dep tăng nhiều nhất 3, còn mỗi lần bạo lực sửa chuyển làm Dep giảm 1. Do đó số lần bạo lực sửa chuyển thực chất bị chặn bởi độ sâu; cộng trên mọi lá sẽ được kết luận trên.

Độ phức tạp này chưa phải đẹp, nhưng độ phức tạp của xây dựng online có thể đạt đến cấp đó.

Hình h. Nếu chèn các nút theo thứ tự chỉ số nút trong ví dụ của bản gốc, độ phức tạp đạt $O(G(T))$.

Khi không yêu cầu online, ta có thể thêm các nút trên trie theo thứ tự FIFO, tức thứ tự duyệt rộng. Khi đó chứng minh độ phức tạp của J. A. Blumer và cộng sự [9, 10] cho thuật toán xây trên xâu cũng áp

dụng cho trie. Chỉ những tổ hợp xâu con cố định mới khiến ta phải bạo lực sửa hàm chuyển [6], gợi ý rằng phần độ phức tạp này không vượt quá $O(|T|)$.

Tóm lại, nếu yêu cầu online, độ phức tạp xây dựng là $O(G(T))$. Nếu cho phép offline, ta có thể xây theo thứ tự FIFO và đạt độ phức tạp $O(|T| |A|)$, đúng với cận dưới đã nêu.

3.6 Ứng dụng

3.6.1 ZJOI 2015: Vùng đất huyền tưởng được chú thần ưu ái

Đề bài đã xuất hiện ở phần lời nói đầu, nên không nhắc lại nữa.

Sau chuyển hóa, bài toán trở thành: đếm số xâu con khác nhau về bản chất của một trie. Bằng phương pháp xây suffix automaton cho trie ở trên, ta thu được một automaton có thể nhận ra mọi hậu tố của trie. Mỗi xâu con của trie chắc chắn là tiền tố của một hậu tố nào đó, giống hệt tính chất khi xây suffix automaton cho một xâu. Vì vậy, với một trạng thái $\Phi(T)_s$, số xâu con khác nhau mà nó đại diện là $|R(s)|$; cộng $|R(s)|$ trên mọi trạng thái sẽ cho đáp án.

Cũng có cách khác. Sau khi xây automaton, trực tiếp đếm có bao nhiêu xâu khác nhau không bị automaton từ chối; điều này tương đương đếm số xâu con khác nhau, hay đếm số đường chuyển khác nhau tồn tại khi bắt đầu từ trạng thái đầu. Xem automaton như một đồ thị có hướng không chu trình, ta chỉ cần quy hoạch động đơn giản trên đồ thị này.

Theo phân tích độ phức tạp ở trên, đề bài không yêu cầu online, nên có thể dùng thuật toán offline với độ phức tạp $O(n|A|)$. Thực ra điều kiện số lá là hằng số rất mạnh; có thể chứng minh automaton xây bằng phương pháp trên có thời gian $O(n)$. Lý do là ta có thể tách riêng đường từ gốc đến từng lá thành các xâu, rồi dùng phân tích độ phức tạp giống trường hợp xâu [9]. Vì điều này không liên quan trực tiếp đến nội dung chính của bài viết, tác giả bỏ qua chứng minh chi tiết.

Đến đây bài toán trong phần lời nói đầu đã được giải quyết trọn vẹn.

Từ bài toán này có thể thấy hầu hết bài toán xâu xử lý được bằng suffix automaton đều có thể mở rộng lên trie: tìm xâu con thứ K nhỏ nhất của trie, tìm xâu con chung dài nhất của nhiều trie, đếm số lần xuất hiện của xâu vòng trên trie, v.v. Cũng có những bài rất dễ trên xâu nhưng phức tạp hơn khi mở rộng lên trie, chẳng hạn đếm số xâu khác nhau mà tiền tố của chúng là một xâu cho trước. Các bài này đều có thể giải sau khi xây suffix automaton cho trie.

3.6.2 So sánh với AC automaton

Trong các thuật toán xâu quen thuộc, AC automaton xây một trie từ m mẫu rồi xây các con trỏ fail, sau đó có thể nhận diện số lần xuất hiện của từng mẫu trong xâu đầu vào. Ta có thể thay thế cách dùng này bằng suffix automaton trên trie: xây trie từ m xâu, rồi xây suffix automaton cho trie đó. Khi đọc xâu cần nhận diện, nếu trạng thái hiện tại $\Phi(T)_s$ không có chuyển nhãn a , ta nhảy tới $fa(\Phi(T)_s)$ và tiếp tục nhận diện. Rõ ràng tổng số lần nhảy không vượt quá độ dài xâu đang nhận diện.

Về độ phức tạp, trie T xây từ m xâu có $G(T)$ không vượt quá tổng độ dài của m xâu, nên độ phức tạp không kém hơn AC automaton. Nhưng thuật toán xây ở đây có thể là online, trong khi thuật toán xây AC automaton chỉ là offline.

3.6.3 Cây parent

Hình i. Bản gốc minh họa việc bổ sung các ký tự trên cạnh của Hình b để nhìn cây parent như một trie của các tiền tố đảo.

Thực chất, ta có thể đảo toàn bộ tiền tố trong trie rồi xây lại thành một trie, tương tự suffix tree của một xâu. Với mỗi trạng thái $\Phi(T)_s$, sắp xếp các con của nó trong cây parent. Cụ thể, nếu

$$Fa(\Phi(T)_x) = \Phi(T)_s,$$

ta sắp xếp các trạng thái $\Phi(T)_x$ theo thứ tự của đoạn

$$x_{1, |x| - \text{Maxl}(R(s))}.$$

Khi đó ta thu được một mảng tiền tố của trie, tức thứ tự sau khi đảo mọi tiền tố. Ta cần duy trì cho $\Phi(T)_s$ một nút $T_v \in \text{Right}(s)$; kết hợp nhân đôi trên cây, có thể nhanh chóng tìm được đoạn $x_{1, |x| - \text{Maxl}(R(s))}$.

Với tính chất này, có thể nhanh chóng xác định trạng thái tương ứng khi thêm một ký tự $a \in A$ vào trước xâu đang nhận diện. Giả sử hiện ở trạng thái $\Phi(T)_s$. Ta tìm nhị phân trong các con của nó trên cây parent để tìm $\Phi(T)_{as}$, hoặc kết luận $\Phi(T)_{as} = \Phi(T)_s$.

3.7 Lời cảm ơn

Xin cảm ơn thầy Qu Yunhua của Trường Yali đã lâu dài hướng dẫn và quan tâm tới việc học tập cũng như cuộc sống của tôi.

Xin cảm ơn cha mẹ đã nuôi dưỡng tôi.

Xin cảm ơn các thầy Zhu Quanmin và Wang Xingming của Trường Yali đã chỉ dạy.

Xin cảm ơn CCF đã cung cấp nền tảng và cơ hội giao lưu.

Xin cảm ơn các huấn luyện viên đội tuyển quốc gia vì sự tận tụy.

Xin cảm ơn các anh Huang Zhixiang của Đại học Thanh Hoa và Huang Yuyang của Đại học Giao thông Thượng Hải đã giúp đỡ tôi.

Xin cảm ơn các bạn Yang Dingcheng, Kuang Zhengfei và Liu Jiancheng của Trường Yali đã đọc duyệt bài viết này.

Xin cảm ơn tất cả những bạn học từng giúp đỡ tôi.

Tài liệu tham khảo của bài viết

- [1] A. V. Aho and M. J. Corasick, Efficient string matching: An aid to bibliographic research.
- [2] R.-S. Boyer and J. S. Moore, A fast string searching algorithm.
- [3] D. E. Knuth, J. H. Morris and V. I. Pratt, Fast pattern-matching in strings.
- [4] P. Weiner, Linear pattern-matching algorithms.
- [5] E. M. McCreight, A space-economical suffix-tree construction algorithm.
- [6] M. Mohri, P. Moreno and E. Weinstein, General Suffix Automaton Construction Algorithm and Space Bounds.
- [7] M. Crochemore, Transducers and repetitions.
- [8] Chen Lijie, *Suffix automaton*.
- [9] J. A. Blumer, Algorithms for the directed acyclic word graph and related structures.
- [10] A. Blumer, J. Blumer, D. Haussler, R. M. McConnell and A. Ehrenfeucht, Complete inverted files for efficient text retrieval and analysis.
- [11] U. Manber and G. Myers, Suffix arrays: A new method for on-line string searches.

- [12] Luo Suijian, *Suffix array: một công cụ mạnh để xử lý xâu.*
- [13] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein, *Introduction to Algorithms.*